

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1403

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Duration :60 Mins
On Class
Total Marks:50

Annexure A

EXPERIMENTS

Code of Conduct:

I Pooja Sree.M (192521403) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Pooja Sree . M
Signature of the student

Annexure A

EXPERIMENTS

Q1. Write a C program to implement a syntax-directed definition for evaluating arithmetic expressions using the following grammar:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow \text{num}$$

Input:

$$3 + 5 * 2$$

The program should display the attribute values and the final value of the expression.

AIM:

To write and execute a C program to implement a syntax-directed definition for evaluating arithmetic expressions using the given grammar and to display the attribute values and final value of the expression.

PROCEDURE:

1. Start the program.
2. Read an arithmetic expression containing numbers, +, and *.
3. Apply the grammar rules to identify F, T, and E.
4. Evaluate multiplication before addition according to the grammar.
5. Display the attribute values for each production.
6. Display the final value of the expression.
7. Stop the program.

CODE:

```
#include <stdio.h>

int main()
{
    int n1, n2, n3;

    int t, e;

    printf("Enter expression: ");

    scanf("%d + %d * %d", &n1, &n2, &n3);

    printf("\nAttribute Values:");

    printf("\nF -> num = %d", n1);

    printf("\nF -> num = %d", n2);

    printf("\nF -> num = %d", n3);

    t = n2 * n3;

    printf("\nT -> T * F = %d * %d = %d", n2, n3, t);

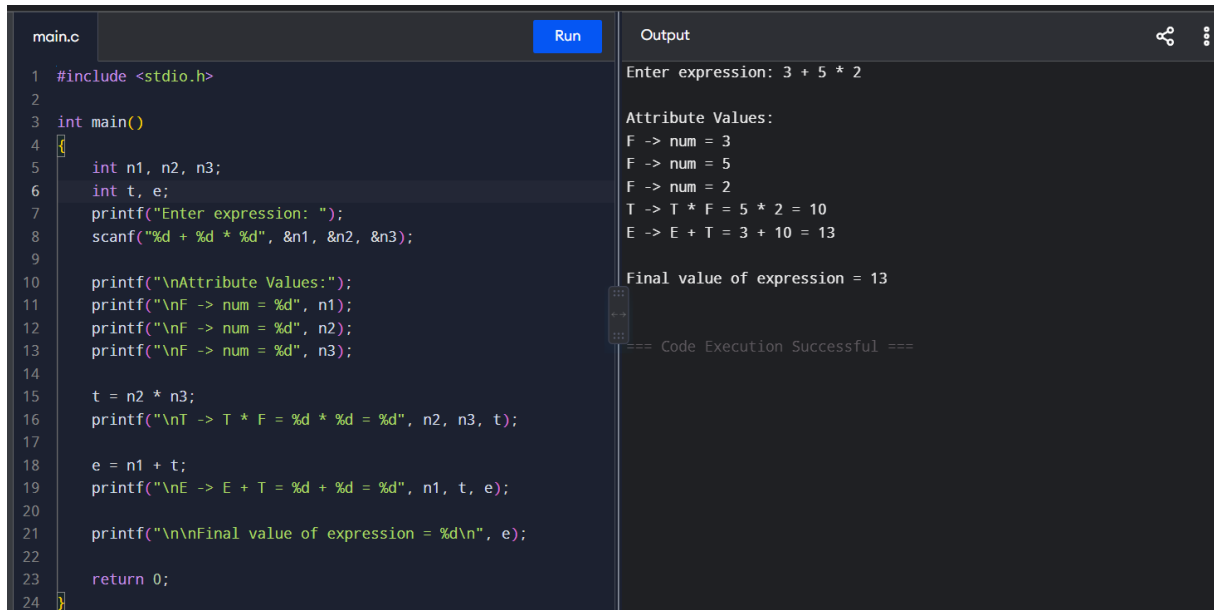
    e = n1 + t;

    printf("\nE -> E + T = %d + %d = %d", n1, t, e);

    printf("\n\nFinal value of expression = %d\n", e);

    return 0;
}
```

OUTPUT:



The screenshot shows a C program in a code editor with a 'Run' button. The code defines variables n1, n2, n3, t, and e, prompts for an expression, and calculates its value using a simple parser. The output window shows the input '3 + 5 * 2' and the resulting attribute values and final expression value.

```
main.c Run Output
1 #include <stdio.h>
2
3 int main()
4 {
5     int n1, n2, n3;
6     int t, e;
7     printf("Enter expression: ");
8     scanf("%d + %d * %d", &n1, &n2, &n3);
9
10    printf("\nAttribute Values:");
11    printf("\nF -> num = %d", n1);
12    printf("\nF -> num = %d", n2);
13    printf("\nF -> num = %d", n3);
14
15    t = n2 * n3;
16    printf("\nT -> T * F = %d * %d = %d", n2, n3, t);
17
18    e = n1 + t;
19    printf("\nE -> E + T = %d + %d = %d", n1, t, e);
20
21    printf("\n\nFinal value of expression = %d\n", e);
22
23    return 0;
24 }
```

Enter expression: 3 + 5 * 2

Attribute Values:
F -> num = 3
F -> num = 5
F -> num = 2
T -> T * F = 5 * 2 = 10
E -> E + T = 3 + 10 = 13

Final value of expression = 13

=== Code Execution Successful ===

RESULT:

Thus, the syntax-directed definition for evaluating the arithmetic expression was successfully implemented. The final value of the expression $3 + 5 * 2$ is 13.

Q2. Write a C program to implement a simple type checker for arithmetic expressions using the following grammar:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow \text{id}$$

Consider the following symbol table:

a : int

b : int

c : float

Input:

$a + b * c$

Determine the type of each subexpression and the final expression.

AIM:

To write a C program to implement a simple type checker for arithmetic expressions and to determine the type of each subexpression and the final expression.

PROCEDURE:

1. Start the program.
2. Define the symbol table containing the identifiers and their types.
3. Read the expression $a + b * c$.
4. Look up the type of each identifier.
5. Determine the type of the multiplication subexpression $b * c$.
6. Determine the type of the addition subexpression $a + (b * c)$.
7. Display the type of each subexpression and the final expression.
8. Stop the program.

CODE:

```
#include <stdio.h>

#include <string.h>

char* getType(char id)
{
    if (id == 'a' || id == 'b')
        return "int";

    else if (id == 'c')
        return "float";

    else
        return "unknown";
}
```

```
}
```

```
char* checkType(char *type1, char *type2)
```

```
{
```

```
    if (strcmp(type1, "float") == 0 ||
```

```
        strcmp(type2, "float") == 0)
```

```
        return "float";
```

```
    else
```

```
        return "int";
```

```
}
```

```
int main()
```

```
{
```

```
    char *typeA, *typeB, *typeC;
```

```
    char *mulType, *finalType;
```

```
    printf("Symbol Table:\n");
```

```
    printf("a : int\n");
```

```
    printf("b : int\n");
```

```
    printf("c : float\n");
```

```
    printf("\nExpression: a + b * c\n");
```

```
    typeA = getType('a');
```

```
    typeB = getType('b');
```

```
    typeC = getType('c');
```

```
    printf("\nType of a = %s", typeA);
```

```
    printf("\nType of b = %s", typeB);
```

```
    printf("\nType of c = %s", typeC);
```

```

mulType = checkType(typeB, typeC);

printf("\n\nType of b * c = %s * %s = %s",
      typeB, typeC, mulType);

finalType = checkType(typeA, mulType);

printf("\n\nType of a + (b * c) = %s + %s = %s",
      typeA, mulType, finalType);

printf("\n\nFinal expression type = %s\n", finalType);

return 0;
}

```

OUTPUT:

main.c	Run	Output
<pre> 24 { 30 printf("b : int\n"); 31 printf("c : float\n"); 32 33 printf("\nExpression: a + b * c\n"); 34 35 typeA = getType('a'); 36 typeB = getType('b'); 37 typeC = getType('c'); 38 39 printf("\nType of a = %s", typeA); 40 printf("\nType of b = %s", typeB); 41 printf("\nType of c = %s", typeC); 42 mulType = checkType(typeB, typeC); 43 44 printf("\n\nType of b * c = %s * %s = %s", 45 typeB, typeC, mulType); 46 47 finalType = checkType(typeA, mulType); 48 49 printf("\n\nType of a + (b * c) = %s + %s = %s", 50 typeA, mulType, finalType); 51 52 printf("\n\nFinal expression type = %s\n", finalType); 53 </pre>		<pre> Symbol Table: a : int b : int c : float Expression: a + b * c Type of a = int Type of b = int Type of c = float Type of b * c = int * float = float Type of a + (b * c) = int + float = float Final expression type = float === Code Execution Successful === </pre>

RESULT:

Thus, the simple type checker was successfully implemented. The type of $b * c$ is float, and the final expression $a + b * c$ has type float.

Q3. Write a C program to determine whether two type expressions are equivalent.

Consider the following type expressions:

T1 = array(10, int)

T2 = array(10, int)

The program should display:

Equivalent or Not Equivalent .

AIM:

To write a C program to determine whether two given type expressions are equivalent or not.

PROCEDURE:

1. Start the program.
2. Store the size and base type of T1.
3. Store the size and base type of T2.
4. Compare the array sizes.
5. Compare the element types.
6. If both are the same, display Equivalent.
7. Otherwise, display Not Equivalent.
8. Stop the program.

CODE:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Type
```

```
{
```

```
int size;

char datatype[20];

};

int main()

{

    struct Type T1, T2;

    T1.size = 10;

    strcpy(T1.datatype, "int");

    T2.size = 10;

    strcpy(T2.datatype, "int");

    printf("T1 = array(%d, %s)\n",

        T1.size, T1.datatype);

    printf("T2 = array(%d, %s)\n",

        T2.size, T2.datatype);

    if (T1.size == T2.size &&

        strcmp(T1.datatype, T2.datatype) == 0)

    {

        printf("\nEquivalent\n");

    }

    else
```

```

{

    printf("\nNot Equivalent\n");

}

return 0;

}

```

OUTPUT:

The screenshot shows a C program in a code editor with a 'Run' button. The code defines a struct 'Type' with 'int size' and 'char datatype[20]'. In the 'main' function, two struct variables 'T1' and 'T2' are created. Both are initialized with 'size = 10' and 'datatype = "int"'. The program then prints the details of both arrays. The output window shows the printed information for both arrays, followed by the word 'Equivalent' and a success message '=== Code Execution Successful ==='.

```

main.c Run Output
1 #include <stdio.h>
2 #include <string.h>
3
4 struct Type
5 {
6     int size;
7     char datatype[20];
8 };
9
10 int main()
11 {
12     struct Type T1, T2;
13
14     T1.size = 10;
15     strcpy(T1.datatype, "int");
16
17     T2.size = 10;
18     strcpy(T2.datatype, "int");
19
20     printf("T1 = array(%d, %s)\n",
21           T1.size, T1.datatype);
22
23     printf("T2 = array(%d, %s)\n",
24           T2.size, T2.datatype);
25
26     printf("Equivalent\n");
27
28     return 0;
29 }

```

Output

```

T1 = array(10, int)
T2 = array(10, int)

Equivalent

=== Code Execution Successful ===

```

RESULT:

Thus, the two type expressions `array(10, int)` and `array(10, int)` were compared successfully and found to be Equivalent.

Q4. Write a C program to construct and display the syntax tree for the following expression using the given grammar:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$F \rightarrow id$

Input:

$a + b * c$

Display the constructed syntax tree.

AIM:

To write and execute a C program to construct and display the syntax tree for the arithmetic expression $a + b * c$ using the given grammar.

PROCEDURE:

1. Start the program.
2. Create nodes for the identifiers a, b, and c.
3. Create a * node.
4. Make b and c the left and right children of *.
5. Create a + node.
6. Make a the left child and * the right child of +.
7. Display the constructed syntax tree.
8. Stop the program.

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node
```

```
{
```

```
    char data;
```

```
    struct Node *left;
```

```
    struct Node *right;

};

struct Node* createNode(char data)

{

    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = data;

    newNode->left = NULL;

    newNode->right = NULL;

    return newNode;

}

void displayTree(struct Node *root, int space)

{

    int i;

    if (root == NULL)

        return;

    space = space + 5;

    displayTree(root->right, space);

    printf("\n");

    for (i = 5; i < space; i++)
```

```
printf(" ");
```

```
printf("%c\n", root->data);
```

```
displayTree(root->left, space);
```

```
}
```

```
int main()
```

```
{
```

```
    struct Node *a;
```

```
    struct Node *b;
```

```
    struct Node *c;
```

```
    struct Node *multiply;
```

```
    struct Node *root;
```

```
    a = createNode('a');
```

```
    b = createNode('b');
```

```
    c = createNode('c');
```

```
    multiply = createNode('*');
```

```
    multiply->left = b;
```

```
    multiply->right = c;
```

```
    root = createNode('+');
```

```
    root->left = a;
```

```

    root->right = multiply;

    printf("Syntax Tree for a + b * c:\n");

    displayTree(root, 0);

    return 0;

}

```

OUTPUT:

The screenshot shows a C program in a code editor with a 'Run' button. The code defines a 'struct Node' with 'data', 'left', and 'right' pointers. It includes functions 'createNode' and 'displayTree'. The output window shows the text 'Syntax Tree for a + b * c:' followed by a tree diagram where '+' is the root, 'a' is its left child, and '*' is its right child. '*' has 'b' as its left child and 'c' as its right child. The output concludes with '=== Code Execution Successful ==='.

```

main.c Run Output
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     char data;
7     struct Node *left;
8     struct Node *right;
9 };
10
11 struct Node* createNode(char data)
12 {
13     struct Node *newNode;
14     newNode = (struct Node*)malloc(sizeof(struct Node));
15     newNode->data = data;
16     newNode->left = NULL;
17     newNode->right = NULL;
18     return newNode;
19 }
20
21 void displayTree(struct Node *root, int space)
22 {
23
24
25

```

Syntax Tree for a + b * c:

```

      c
     *
    b
   +
  a

```

=== Code Execution Successful ===

RESULT:

Thus, the syntax tree for the expression $a + b * c$ was successfully constructed and displayed, with $+$ as the root and $*$ as its right child.

Q5. Write a C program to demonstrate the propagation of inherited and synthesized attributes using the following grammar:

$D \rightarrow T L$

$T \rightarrow \text{int}$

$L \rightarrow \text{id } L'$

$L' \rightarrow , \text{id } L' \mid \epsilon$

Input:

int a,b,c

Display the type assigned to each identifier.

AIM:

To write and execute a C program to demonstrate the propagation of inherited and synthesized attributes and assign the declared type to each identifier.

PROCEDURE:

1. Start the program.
2. Read the declaration int a,b,c.
3. Identify int using the production $T \rightarrow \text{int}$.
4. Pass the type int as an inherited attribute to the identifier list.
5. Identify each identifier using $L \rightarrow \text{id } L'$.
6. Continue processing identifiers using $L' \rightarrow , \text{id } L'$.
7. Assign int to every identifier.
8. Display the type assigned to each identifier.
9. Stop the program.

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char input[100];
```

```
    char id[20];
```

```
    int i = 3;
```

```
    int j;
```

```

printf("Enter declaration: ");

fgets(input, sizeof(input), stdin);

printf("\nType assigned to each identifier:\n");

while (input[i] != '\0' && input[i] != '\n')

{

    /* Skip spaces and commas */

    while (input[i] == ' ' || input[i] == ',')

        i++;

    if (input[i] == '\0' || input[i] == '\n')

        break;

    j = 0;

    while (input[i] != '\0' &&

        input[i] != '\n' &&

        input[i] != ';' &&

        input[i] != ')')

    {

        id[j] = input[i];

        j++;

        i++;

```

```
}
```

```
id[j] = '\0';
```

```
if (j > 0)
```

```
printf("%s : int\n", id);
```

```
}
```

```
return 0;
```

```
}
```

OUTPUT:

The screenshot shows a C code editor with a file named `main.c`. The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     char input[100];
6     char id[20];
7     int i = 3;
8     int j;
9
10    printf("Enter declaration: ");
11    fgets(input, sizeof(input), stdin);
12
13    printf("\nType assigned to each identifier:\n");
14
15    while (input[i] != '\0' && input[i] != '\n')
16    {
17        while (input[i] == ' ' || input[i] == ',')
18            i++;
19
20        if (input[i] == '\0' || input[i] == '\n')
21            break;
22
23        j = 0;
24
25        while (input[i+j] != '\0' &&
```

The output window on the right shows the following text:

```
Enter declaration: int a,b,c

Type assigned to each identifier:
a : int
b : int
c : int

=== Code Execution Successful ===
```

RESULT:

Thus, the propagation of attributes was successfully demonstrated, and the type `int` was assigned to all the identifiers `a`, `b`, and `c`.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation